

Complete DSA Learning Order

Topic Content Compilation

Section 3 — Linked Lists (3.1 – 3.5)

June 30, 2026

Section 3 — Linked Lists

3.1 — Singly linked list — structure, traversal, insertion, deletion

1. PLAIN-LANGUAGE GIST

A singly linked list is a chain of nodes where each node holds a value and a pointer to the next node — there's no contiguous memory block like an array, just nodes scattered in memory, connected by pointers. It exists because arrays are expensive to insert into or delete from in the middle (everything has to shift), while a linked list can splice nodes in or out with just a couple of pointer changes, at the cost of losing instant index-based access.

2. REAL-WORLD ANALOGY

Think of a treasure hunt where each clue tells you the location of the next clue, rather than a numbered map where you can jump straight to clue #7. You can't skip ahead — you must follow the chain of clues one at a time — but inserting a brand-new clue into the middle of the hunt is trivial: you just change what one clue points to, without reprinting or renumbering anything else.

3. CONNECT TO PRIOR KNOWLEDGE

This is the structural payoff of everything 0.7's recursion fundamentals built toward — linked lists are the canonical example of structural recursion (a list is either empty, or a node plus a smaller list), and 0.4's call-stack-as-linked-structure intuition transfers directly. It's also the first data structure in the roadmap that deliberately trades away 1.1's $O(1)$ array indexing in exchange for cheap insertion/deletion.

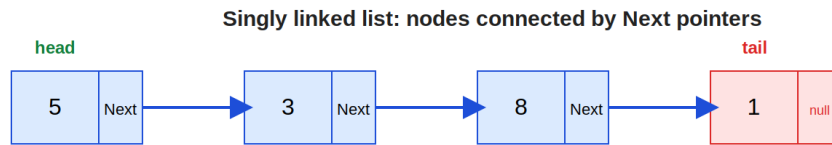
4. CORE MECHANISM (Simple → Intermediate)

Simple model: A node is a small object holding a Value and a Next pointer (reference to another node, or null if it's the last node). A linked list is just a reference to the first node (head) — everything else is reached by following Next pointers one at a time.

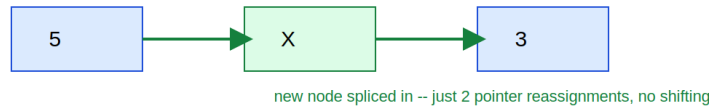
Next layer — why traversal, insertion, and deletion behave so differently from arrays:

- **Traversal** is $O(n)$ and must be sequential — there's no way to jump to “the 5th node” without walking through the first 4, unlike 1.1's array indexing. This is the fundamental tradeoff: you give up $O(1)$ random access entirely.
- **Insertion** at a known position is $O(1)$ if you already have a reference to the node before the insertion point — you just create a new node, point its Next at the old next node, and point the prior node's Next at the new node. No shifting, unlike an array insertion (which is $O(n)$ due to shifting elements).
- **Deletion** is similarly $O(1)$ given the prior node — you simply skip over the node being removed by pointing the prior node's Next directly at the node after the one being deleted. The deleted node becomes unreachable and is eventually garbage collected.
- **The catch:** finding “the node before the insertion/deletion point” in a singly linked list requires traversing from the head, which is $O(n)$ unless you already have a pointer to that exact prior node (e.g., you're already there during a traversal). This is why linked list problems often track a prev pointer alongside curr during traversal — exactly the pattern formalized in 3.4's reversal.

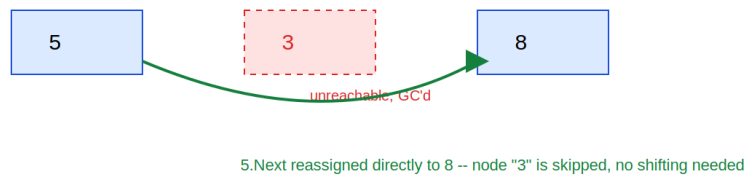
5. VISUAL REPRESENTATION



Insertion (new node "X" between 5 and 3): $O(1)$ if you have the prior node



Deletion (remove "3"): skip over it by relinking 5 -> 8 directly



The top section shows the basic structure (each node's Next pointing to the following node, tail pointing to null). The insertion diagram shows splicing a new node "X" between two existing nodes using only two pointer reassignments — no shifting. The deletion diagram shows relinking around a removed node, leaving it unreachable for garbage collection.

6. WHEN TO USE / WHEN NOT TO USE

Use a singly linked list when:

- You need frequent insertions/deletions at the front or at a position you're already iterating toward, and don't need random access by index.
- You're implementing another structure on top of it (a stack, via 4.1, is naturally a singly linked list with insert/remove only at the head).

Wrong choice when:

- You need to access elements by index frequently — that's $O(n)$ on a linked list versus $O(1)$ on an array (1.1); use an array or `List<T>` instead.
- You need to traverse backward — a singly linked list only goes forward; if backward traversal is common, a doubly linked list (3.2) is the right structure.
- Cache locality / raw iteration speed matters a lot — nodes are scattered in memory (no contiguous block), so even though traversal is " $O(n)$ " just like an array scan, it tends to be slower in practice due to cache misses, echoing 1.1's deeper-knowledge point about array cache locality.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — node definition and basic traversal:

```

class ListNode {
    public int Value;
    public ListNode Next;
    public ListNode(int value) { Value = value; }
}

void PrintList(ListNode head) {
    ListNode curr = head;
    while (curr != null) {          // O(n) -- must walk node by node
        Console.Write(curr.Value + " ");
        curr = curr.Next;
    }
}

```

INTERMEDIATE — insertion after a given node ($O(1)$, given the node reference):

```

void InsertAfter(ListNode node, int value) {
    var newNode = new ListNode(value);
    newNode.Next = node.Next;    // new node points to what node used to point to
    node.Next = newNode;        // node now points to the new node
}

```

Order matters here: you must save `node.Next` into the new node before overwriting `node.Next` — reversing this order would lose the rest of the list, the exact same “save before overwrite” discipline formalized in 3.4’s reversal.

ADVANCED — deletion by value, requiring a prev pointer tracked during traversal, including the head-removal edge case:

```

ListNode DeleteValue(ListNode head, int value) {
    if (head == null) return null;

    if (head.Value == value) {      // special case: deleting head itself
        return head.Next;          // new head is simply old second node
    }

    ListNode prev = head, curr = head.Next;
    while (curr != null) {
        if (curr.Value == value) {
            prev.Next = curr.Next;  // skip over curr entirely
            return head;
        }
        prev = curr;
        curr = curr.Next;
    }
    return head;    // value not found, list unchanged
}

```

The head-removal special case is the most commonly missed edge case in linked list problems: every other deletion needs a prev pointer to relink around the target, but removing the head node has no prev at all, so it must return a different head entirely rather than doing an in-place relink.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, many linked list problems become dramatically simpler using a dummy/sentinel head node — a placeholder node before the real head that absorbs the “deleting/inserting at the head” special case shown above, letting you write one uniform piece of relinking logic instead of branching for the head separately. There’s also a memory-layout contrast worth internalizing: arrays get fast iteration from contiguous memory and CPU cache prefetching (1.1, 1.8’s deeper knowledge), while linked lists trade that away for $O(1)$ splice operations — this is a textbook example of there being no universally “best” data structure, only

the right tradeoff for the access pattern a problem demands.

9. COMMON MISCONCEPTIONS AND MISTAKES

A very common mistake is overwriting a node's Next pointer before saving the value you still need from it — exactly the “save next before reassigning” trap shown in the intermediate example, and the single most frequent source of “lost the rest of the list” bugs. Another is forgetting the head-removal special case during deletion — code that works perfectly for every other node silently breaks (or throws a null-reference exception) when asked to delete the head, because there's no prev node to relink. A third: assuming linked list traversal is “just as fast” as array traversal because both are $O(n)$ — the Big-O classification is identical, but cache-unfriendly scattered memory access typically makes linked list traversal measurably slower in practice, a distinction Big-O notation intentionally doesn't capture.

10. SELF-CHECK

- Why is insertion into a singly linked list $O(1)$ given a reference to the prior node, but effectively $O(n)$ if you only have the value to insert after, not a direct node reference?
- Walk through, out loud, why you must save node.Next into the new node's Next before overwriting node.Next during insertion — what specifically breaks if you do it in the opposite order?
- Why does deleting the head node require special-case handling that no other node's deletion needs?

3.2 — Doubly linked list

1. PLAIN-LANGUAGE GIST

A doubly linked list is a singly linked list where every node also holds a pointer back to the previous node, not just forward to the next one. It exists because some operations — deleting a known node, traversing backward, or moving in either direction — become $O(1)$ or much simpler with a backward pointer, at the cost of slightly more memory per node and more pointers to keep consistent.

2. REAL-WORLD ANALOGY

If a singly linked list's treasure hunt clues only tell you where the next clue is, a doubly linked list's clues tell you where both the next and the previous clue are. If you're standing at clue #5 and need to remove it from the hunt, you don't need to walk back from the start to find clue #4 — clue #5 already told you exactly where it is.

3. CONNECT TO PRIOR KNOWLEDGE

This directly extends 3.1's node structure by adding one more pointer (Prev), and it directly resolves 3.1's biggest limitation — needing a separately-tracked prev variable during traversal just to delete a node — by baking that backward reference into the structure itself.

4. CORE MECHANISM (Simple → Intermediate)

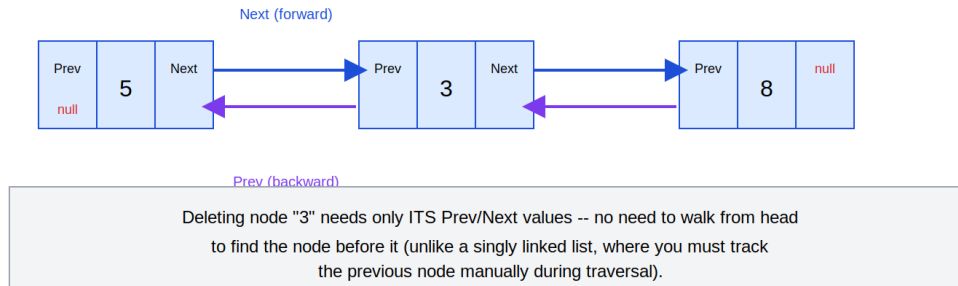
Simple model: Each node holds Value, Next, and Prev. The list itself often tracks both Head and Tail references, since you can now usefully start traversal from either end.

Next layer — what the extra pointer actually buys you, and what it costs:

- **$O(1)$ deletion given just the node itself** (not even a prev reference) — since the node already knows its own Prev, you can relink `node.Prev.Next = node.Next` and `node.Next.Prev = node.Prev` directly, no traversal needed to find the prior node. This is the direct fix for 3.1's deletion limitation.
- **Backward traversal** becomes possible — starting from Tail and following Prev pointers, useful for “process in reverse” without needing to reverse the list first (3.4) or use extra space to store a reversed copy.
- **The cost:** every insertion and deletion must correctly update two pointers per affected node instead of one, which doubles the number of relinking steps and roughly doubles the opportunities for a subtle off-by-one or forgotten-pointer bug. Every Next reassignment now has a matching Prev reassignment that must also happen, in the correct order, or the list becomes internally inconsistent (e.g., `A.Next == B` but `B.Prev != A`).
- **Memory overhead:** each node carries one extra pointer's worth of memory — usually negligible, but worth mentioning if asked about tradeoffs, tying back to 0.4's space-complexity thinking.

5. VISUAL REPRESENTATION

Doubly linked list: each node has Prev AND Next pointers



The diagram shows three nodes with both Next (forward, blue) and Prev (backward, purple) pointers. The key practical payoff highlighted at the bottom: deleting the middle node needs only that node's own Prev/Next values — no traversal from the head required, unlike the singly linked list case in 3.1.

6. WHEN TO USE / WHEN NOT TO USE

Use a doubly linked list when:

- You need $O(1)$ deletion given just a node reference (no separately tracked prev needed) — common in LRU-cache-style structures where you hold direct references to nodes you might need to remove.
- You need efficient traversal in both directions, or need to access the tail as easily as the head.

Wrong choice when:

- You only ever traverse forward and never need to delete by direct node reference — the extra Prev pointer's bookkeeping cost buys you nothing in that case, and a singly linked list (3.1) is simpler and slightly more memory-efficient.
- Memory is extremely tight and the list will be very large — the extra pointer per node adds up at scale, even though it's a constant factor, not a complexity-class change.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — node definition with both pointers:

```
class DoublyNode {
    public int Value;
    public DoublyNode Next;
    public DoublyNode Prev;
    public DoublyNode(int value) { Value = value; }
}
```

INTERMEDIATE — $O(1)$ deletion given just the node itself (the key payoff over 3.1):

```
void DeleteNode(DoublyNode node) {
    if (node.Prev != null) node.Prev.Next = node.Next;
    if (node.Next != null) node.Next.Prev = node.Prev;
    // node itself is now unreachable from the list -- no traversal needed
}
```

Compare this directly to 3.1's DeleteValue — there, you needed a prev variable tracked during an $O(n)$ traversal; here, the node already carries everything needed to relink around itself in $O(1)$, as long as you already have a reference to it.

ADVANCED — insertion that correctly updates all four affected pointers, including the edge case of inserting at the tail:

```

void InsertAfter(DoublyNode node, int value) {
    var newNode = new DoublyNode(value);
    newNode.Prev = node;
    newNode.Next = node.Next;

    if (node.Next != null) {           // node was NOT the tail
        node.Next.Prev = newNode;
    }
    node.Next = newNode;
    // if node WAS the tail, the list's Tail reference would also need
    // updating here -- easy to forget since it's not a node pointer
}

```

This shows the doubled bookkeeping directly: a singly linked list insertion (3.1) touches 2 pointers; this touches up to 4 (`newNode.Prev`, `newNode.Next`, `node.Next.Prev`, `node.Next`), plus a possible external update to the list's Tail reference if the insertion happens at the very end — a detail that's easy to forget because it's not a node-internal pointer at all.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, doubly linked lists are the backbone of several well-known higher-level structures — .NET's own `LinkedList<T>` is a doubly linked list internally, and the classic LRU (Least Recently Used) cache combines a doubly linked list (for $O(1)$ “move this item to the front” and $O(1)$ eviction from the back) with a `Dictionary<TKey, DoublyNode>` (for $O(1)$ lookup of which node corresponds to a given key) — a genuinely common “combine two structures” interview pattern that directly builds on both Section 2's hashing and this topic. There's also a circular variant (a circular doubly linked list, where the tail's Next points back to the head and the head's Prev points to the tail) used in some specialized scheduling and buffering systems, eliminating null checks entirely at the cost of needing a sentinel/dummy node to mark “start” unambiguously.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is updating only the Next pointers during insertion/deletion and forgetting the matching Prev updates (or vice versa) — this is the single most frequent bug in doubly linked list code, producing a list that looks correct when traversed forward but is silently broken when traversed backward (or when a later operation relies on Prev being accurate). Another is forgetting to update the list's external Tail (or Head) reference when an operation happens to affect the very first or last node — these aren't node-internal pointers, so they're easy to overlook even when all the node-level pointer updates are correct. A third: assuming a doubly linked list is strictly “better” than a singly linked list because it can do more — it carries real memory and bookkeeping overhead, and using one when you only ever need forward traversal and head-only insertion is unnecessary complexity for no benefit.

10. SELF-CHECK

- Why can a doubly linked list delete a given node in $O(1)$ without needing a separately-tracked prev pointer, while a singly linked list cannot?
- In the insertion example, why are there potentially four pointers to update instead of the two a singly linked list insertion needs — name each one and what it's responsible for.
- Why is it easy to forget updating the list's Tail reference during certain insertions/deletions, and what kind of bug does forgetting it cause?

3.3 — Fast and slow pointers (cycle detection, finding middle)

1. PLAIN-LANGUAGE GIST

Fast and slow pointers means walking two pointers through a linked list at different speeds (typically one step vs. two steps per iteration) to extract structural information — like the middle of the list, or whether the list loops back on itself — without needing to know the list's length in advance or use extra space to store positions.

2. REAL-WORLD ANALOGY

Picture two runners on a circular track, one running twice as fast as the other. If the track is actually a loop (not a straight line that ends), the faster runner will eventually catch up and lap the slower one from behind — they're guaranteed to meet at some point on the track. If the track is a straight line with a clear finish, the faster runner simply finishes first, and by the time they do, the slower runner is reliably about halfway through.

3. CONNECT TO PRIOR KNOWLEDGE

This is a direct adaptation of 1.2's two-pointer technique to linked lists — instead of array indices moving toward each other or scanning forward together at the same rate, here the two pointers move at different rates on a structure that can only be traversed sequentially (3.1), which is exactly why this specific variant exists: linked lists don't support 1.1's $O(1)$ indexing, so you can't just compute “the middle index” directly the way you could on an array.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: Initialize two pointers, slow and fast, both starting at the head. Each iteration: slow moves one step ($\text{slow} = \text{slow.Next}$), fast moves two steps ($\text{fast} = \text{fast.Next.Next}$). Keep going until fast reaches the end (or a stopping condition specific to the problem).

Next layer — why this finds the middle, and why it detects cycles:

- **Finding the middle:** since fast moves twice as fast as slow, by the time fast has traveled the full length of the list, slow has only traveled half that distance — landing exactly at the middle. This avoids a two-pass approach (one pass to count the length, a second pass to walk to the middle), doing it in a single pass instead.
- **Cycle detection (Floyd's algorithm):** if the list has no cycle, fast simply reaches null and the loop ends — no cycle exists. If the list does have a cycle, fast can never reach null (it just keeps looping), and because fast gains on slow by exactly one extra step per iteration once both are inside the loop, fast is guaranteed to eventually “lap” slow and land on the exact same node as slow at some iteration — this meeting is only possible if a cycle exists, making it a reliable detection signal.
- **Why this needs no extra space:** the naive way to detect a cycle would be tracking every visited node in a `HashSet<ListNode>` (an application of 2.5's duplicate detection) and checking if you ever revisit one — that works, but costs $O(n)$ space. Fast/slow pointers achieve the same detection in $O(1)$ space, trading a slightly less obvious mechanism for a real space saving, which is exactly the kind of tradeoff worth explicitly mentioning if asked “can you do this with less space?” (0.4's interview signal).

5. VISUAL REPRESENTATION

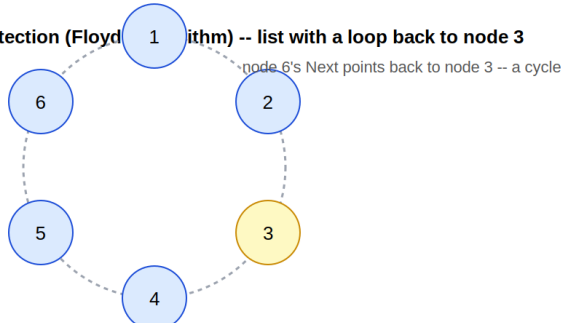
Fast and slow pointers: finding the middle, and detecting a cycle

Finding the middle (list of 6 nodes)



slow moves 1 step, fast moves 2 steps per iteration -- when fast hits the end, slow is at the middle

Cycle detection (Floyd's algorithm) -- list with a loop back to node 3



slow and fast both enter the loop; fast (2x speed) inevitably "laps" slow and they meet at the SAME node -- this meeting is only possible if a cycle exists

The top half traces finding the middle of a 6-node list: by the time fast reaches the end, slow has landed exactly on node 3 (the middle). The bottom half shows a list with a cycle (node 6 looping back to node 3) — both pointers enter the loop, and because fast gains ground every iteration, they're guaranteed to converge on the same node.

6. WHEN TO USE / WHEN NOT TO USE

Use fast/slow pointers when:

- You need the middle (or a specific fractional position) of a linked list in a single pass without knowing its length ahead of time.
- You need to detect whether a linked list contains a cycle, or (a common follow-up) find where the cycle begins, without using extra space for a visited-set.

Wrong choice when:

- You're working with an array instead of a linked list — arrays support $O(1)$ indexing (1.1), so finding the middle is just `arr[arr.Length / 2]` directly; fast/slow pointers solve a problem specific to sequential-access-only structures.
- You need to detect duplicates or revisits in a context where nodes aren't strictly chained in a single sequence (e.g., a general graph, not a simple list) — that's a job for graph traversal with an explicit visited set (Tier 1 §10), since fast/slow pointers rely specifically on the single-chain structure of a linked list.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — finding the middle node (the exact trace above):

```
ListNode FindMiddle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.Next != null) {
        slow = slow.Next;
        fast = fast.Next.Next;
    }
    return slow;    // lands on the middle (or 2nd middle, for even length)
}
```

The `fast != null && fast.Next != null` guard handles both even- and odd-length lists safely — without it, `fast.Next.Next` could throw a null-reference exception on the last step of an odd-length list.

INTERMEDIATE — detecting whether a cycle exists (Floyd's algorithm, the exact trace above):

```
bool HasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.Next != null) {
        slow = slow.Next;
        fast = fast.Next.Next;
        if (slow == fast) return true;    // they met -- a cycle exists
    }
    return false;    // fast reached null -- no cycle
}
```

Note `slow == fast` is a reference comparison (the exact same node object), not a value comparison — two different nodes could coincidentally hold the same Value, but only an actual cycle makes the pointers land on the identical node.

ADVANCED — finding where the cycle begins, a genuinely clever two-phase extension:

```
ListNode FindCycleStart(ListNode head) {
    ListNode slow = head, fast = head;

    // Phase 1: detect the cycle, same as before
    while (fast != null && fast.Next != null) {
        slow = slow.Next;
        fast = fast.Next.Next;
        if (slow == fast) break;
    }
    if (fast == null || fast.Next == null) return null;    // no cycle

    // Phase 2: reset one pointer to head, move BOTH one step at a
    // time -- they meet exactly at the cycle's starting node (a
    // provable property based on distances, recognize and apply it)
    slow = head;
    while (slow != fast) {
        slow = slow.Next;
        fast = fast.Next;
    }
    return slow;
}
```

This is a case where the mechanism (two pointers, different phases) is approachable, but the mathematical proof of why phase 2 lands exactly on the cycle's start is genuinely non-obvious — appropriate to know the pattern exists and apply it correctly, without needing to re-derive the underlying distance proof on the spot.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, the “why does phase 2 work” proof in the advanced example relies on setting up distance equations between the head, the cycle's start, and the meeting point from phase 1 — showing that the distance from the head to the cycle start equals the distance from the phase-1 meeting point to the cycle start (when both pointers move at the same, slower speed). This general technique — using carefully chosen relative speeds between two pointers to extract positional information without extra memory — generalizes beyond linked lists into certain array problems too (e.g., finding a duplicate number in a specific range using the array's values as implicit “next pointers,” a clever disguised application of the exact same cycle-detection idea).

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is comparing node values (`slow.Value == fast.Value`) instead of node references (`slow == fast`) when checking for a cycle meeting point — two distinct nodes can coincidentally share a value, which would produce a false positive; only reference equality correctly signals that the pointers have actually converged on the same node. Another is forgetting the `fast.Next != null` guard (in addition to `fast != null`) in the loop condition — since `fast` moves two steps per iteration, you must verify both the current and next node exist before dereferencing `fast.Next.Next`, or you risk a null-reference exception on lists of even length. A third: assuming the “two-phase” cycle-start-finding trick is something you should derive the proof for live in an interview — it's far more practical to recognize and correctly apply the pattern (and be able to explain its high-level intuition) than to attempt a full from-scratch derivation under time pressure.

10. SELF-CHECK

- Why does moving `slow` one step and `fast` two steps per iteration guarantee `slow` lands on the middle by the time `fast` reaches the end?
- Explain, out loud, why comparing `slow == fast` must be a reference comparison rather than a value comparison for cycle detection to work correctly.
- Why does fast/slow-pointer cycle detection use $O(1)$ space, while the alternative “track every visited node in a `HashSet`” approach uses $O(n)$ — what's the structural reason fast/slow pointers can get away with using no extra memory?

3.4 — Reversing a linked list (iterative and recursive)

1. PLAIN-LANGUAGE GIST

Reversing a linked list means flipping every node's direction so the last node becomes the first and vice versa — and because nodes can only point forward by default (3.1), reversing means re-pointing every single Next pointer to point backward instead, done either iteratively (walking forward, redirecting pointers as you go) or recursively (trusting that the rest of the list reverses itself, then fixing up one link).

2. REAL-WORLD ANALOGY

Picture a one-way line of people, each person's hand on the shoulder of the person in front of them. To reverse the line, each person needs to turn around and put their hand on the shoulder of the person who used to be behind them — but you have to do this carefully, one person at a time, making sure you remember who was originally in front before you sever that connection, or you'll lose track of the rest of the line.

3. CONNECT TO PRIOR KNOWLEDGE

The iterative version is a direct, careful application of 3.1's “save the next pointer before overwriting” discipline, repeated across the whole list. The recursive version is a direct application of 0.7's “leap of faith” mindset — trust that reversing the smaller sublist already works correctly, and figure out just the one extra step needed to attach the current node onto that already-reversed result.

4. CORE MECHANISM (Simple → Intermediate)

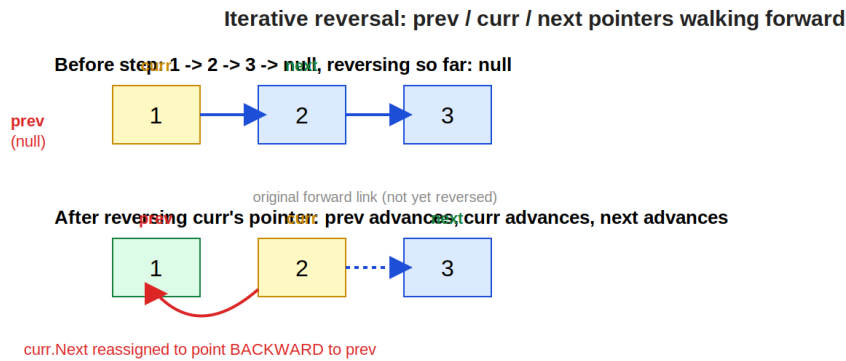
Simple model — iterative: Track three pointers as you walk forward: prev (the new, already-reversed portion, initially null), curr (the node currently being processed), and next (saved before you overwrite curr.Next, so you don't lose the rest of the list). At each step: save next, point curr.Next back to prev, then shift all three pointers forward by one.

Simple model — recursive: Apply 0.7's leap of faith directly: assume Reverse(head.Next) correctly reverses everything after the current head, returning the new head of that reversed sublist. Your job is just to attach the original head onto the end of that already-reversed sublist, then return the new overall head.

Next layer — the key insight that makes both versions click:

- In the iterative version, the critical ordering is: save next before reassigning curr.Next, exactly the trap flagged in 3.1's insertion example, just now happening on every single node in a loop instead of once.
- In the recursive version, the base case is reaching null or a single-node list (already “reversed” trivially) — and the recursive case's one extra step is: after Reverse(head.Next) returns the new head of the reversed remainder, you need head.Next.Next = head (pointing the original second node's Next back at head) and head.Next = null (since head is now the new tail, it must point to nothing).
- Both versions are $O(n)$ time; the iterative version is $O(1)$ space, while the recursive version is $O(n)$ space due to the call stack (0.4's recursion-space trap) — a tradeoff explicitly worth stating if asked.

5. VISUAL REPRESENTATION



Each iteration: save next BEFORE overwriting curr.Next (or you lose the rest of the list),
point curr.Next back to prev, then shift all three pointers one step forward.
Repeat until curr is null -- prev is then the new head of the reversed list.

The diagram traces one step of the iterative version: prev, curr, and next are shown before and after a single iteration, with curr.Next being redirected backward to prev and all three pointers shifting forward afterward.

6. WHEN TO USE / WHEN NOT TO USE

Use the iterative version when:

- You want $O(1)$ space — the default preference in most interview contexts, especially for very long lists where recursion depth could risk a stack overflow (0.4, 0.7's deeper-knowledge warnings about C#'s lack of guaranteed tail-call optimization).

Use the recursive version when:

- You're explicitly asked for a recursive solution, or the problem is naturally framed recursively (e.g., reversing only part of a list, where the recursive structure maps cleanly onto sublists) — but be ready to mention the $O(n)$ space tradeoff unprompted.

Wrong choice / traps to avoid:

- Using recursion on a list whose length isn't bounded and could be very large — risks a real `StackOverflowException`, not just “slower,” echoing 0.7's explicit warning about deep, non-tail-call-optimized recursion in C#.
- Forgetting to set the original head's Next to null in the recursive version — without this, the original head (now the new tail) would still point forward into what's now a fully-reversed structure, silently creating a cycle or a logically broken list.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — iterative reversal (the exact diagram above):

```
ListNode ReverseIterative(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.Next; // SAVE before overwriting
        curr.Next = prev;          // redirect backward
        prev = curr;               // shift prev forward
        curr = next;              // shift curr forward
    }
    return prev; // prev is the new head once curr becomes null
}
```

INTERMEDIATE — recursive reversal, applying 0.7's leap of faith directly:

```

ListNode ReverseRecursive(ListNode head) {
    if (head == null || head.Next == null) return head; // base case

    ListNode newHead = ReverseRecursive(head.Next); // trust this works
    head.Next.Next = head; // attach head onto end of reversed rest
    head.Next = null; // head is now tail -- must point to nothing
    return newHead; // the head of the fully reversed list
}

```

Tracing this by hand on a 3-node list (0.8's skill) is genuinely worth doing once: the recursive calls go all the way down to the last node first, then each call, as it returns, performs exactly one relinking step on its way back up — directly mirroring 0.8's “unwinding” visualization.

ADVANCED — reversing only a sublist between positions left and right (combining iterative reversal with careful boundary relinking):

```

ListNode ReverseBetween(ListNode head, int left, int right) {
    if (head == null || left == right) return head;

    var dummy = new ListNode(0) { Next = head }; // avoids head edge case
    ListNode beforeLeft = dummy;
    for (int i = 1; i < left; i++) beforeLeft = beforeLeft.Next;

    ListNode curr = beforeLeft.Next, prev = null;
    for (int i = 0; i < right - left + 1; i++) {
        ListNode next = curr.Next;
        curr.Next = prev;
        prev = curr;
        curr = next;
    }

    beforeLeft.Next.Next = curr; // old sublist head connects to after
    beforeLeft.Next = prev; // beforeLeft connects to new sub head
    return dummy.Next;
}

```

This combines the core iterative reversal loop with the dummy/sentinel node technique previewed in 3.1's deeper knowledge — using a placeholder node before the real head means “reversing starting at the head” doesn't need separate special-case logic from “reversing starting somewhere in the middle.”

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, linked list reversal is sometimes required in groups of k (reverse every k nodes, then move to the next group) — a more involved combination of the iterative technique here with careful boundary bookkeeping between groups, often considered a genuinely hard variant precisely because of how many pointers must be tracked correctly across group boundaries simultaneously. There's also a connection worth noting to functional programming: recursive reversal's “leap of faith” structure is exactly how immutable, persistent linked lists are reversed in functional languages, where mutation isn't allowed at all — the recursive shape of the solution stays meaningful even outside of languages with mutable references like C#.

9. COMMON MISCONCEPTIONS AND MISTAKES

The most common mistake is exactly the one flagged in section 4: overwriting `curr.Next` before saving its original value into `next`, which permanently severs the connection to the rest of the list — once this happens, there's no way to recover the lost nodes, making it a silent, severe bug rather than an exception. Another is forgetting to null out the original head's `Next` pointer in the recursive version, which can create a subtle cycle or leave a logically incorrect structure that looks fine until you try to traverse to the new tail. A third: defaulting to the recursive solution

without mentioning its $O(n)$ space cost — in an interview, proactively noting “the iterative version achieves the same result in $O(1)$ space” when you present a recursive solution demonstrates the same kind of dual-axis awareness (time and space) that 0.4 emphasized as a strong signal.

10. SELF-CHECK

- Walk through, out loud, what would go wrong if you wrote `curr.Next = prev` before saving `next = curr.Next` in the iterative version.
- In the recursive version, why is the line `head.Next = null` necessary — what specifically would be wrong with the final list if it were omitted?
- Why is the iterative version generally preferred over the recursive version for very long lists, beyond just “it’s a personal style preference”?

3.5 — Merging linked lists

1. PLAIN-LANGUAGE GIST

Merging two (already sorted) linked lists means combining them into a single sorted list by repeatedly comparing the current front of each list and splicing the smaller one onto the result — without creating any new nodes, just relinking the existing ones into a new order. It exists because this exact “merge two sorted sequences” operation is the core subroutine inside merge sort (Tier 1 §7), making it worth mastering as a standalone skill first.

2. REAL-WORLD ANALOGY

Picture two people each holding a sorted stack of numbered cards, face up, lowest on top. To merge them into one sorted stack, you repeatedly glance at both people's top cards, take whichever is lower, and place it onto a new growing stack — continuing until one person runs out, at which point you simply take the rest of the other person's stack as-is, since it's already sorted.

3. CONNECT TO PRIOR KNOWLEDGE

This combines 3.1's relinking discipline (no new nodes, just pointer changes) with a comparison-driven decision loop reminiscent of 1.2's two-pointer technique — except instead of array indices, you're comparing the current front nodes of two separate chains, advancing whichever pointer “loses” the comparison.

4. CORE MECHANISM (Simple → Intermediate)

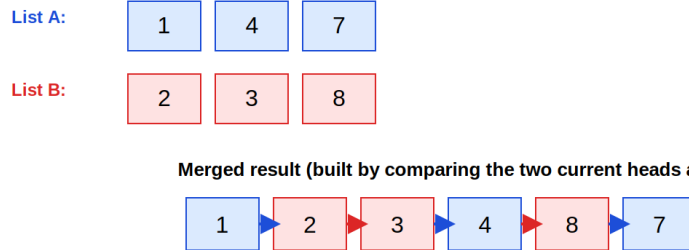
Simple model: Maintain a pointer into each list (a and b, starting at each list's head) and a “tail of the result so far” pointer. At each step, compare a.Value and b.Value; splice whichever is smaller onto the result, and advance that list's pointer. Repeat until one list runs out, then attach the entire remainder of the other list directly (no need to keep comparing one-by-one once the other list is empty).

Next layer — the dummy node trick, and why it matters here specifically:

- Just like 3.4's ReverseBetween example, merging benefits enormously from a dummy/sentinel node at the very start of the result — without it, you'd need special-case logic to handle “what's the very first node of the merged result” separately from every subsequent splice, since there's no “previous node” to attach the first real node to. With a dummy node, every splice (including the very first) follows identical logic: attach to tail.Next, then advance tail.
- No new nodes are created — this is a relinking operation, exactly like reversal (3.4), just operating across two chains instead of one. This matters for space complexity: the merge itself is $O(1)$ extra space (beyond the dummy node), with the $O(n+m)$ time cost coming purely from walking through both lists once.
- This is precisely the merge step of merge sort (a preview of Tier 1 §7) — once you can merge two already-sorted lists efficiently, merge sort's strategy becomes “recursively split a list in half, sort each half, then merge the two sorted halves back together using exactly this technique.”

5. VISUAL REPRESENTATION

Merging two sorted linked lists by repeatedly picking the smaller head



At each step, compare the two CURRENT heads (one from each list) and splice the smaller one onto the result, advancing only that list's pointer. No new nodes are created -- existing nodes are relinked, just like reversal (3.4) reused existing nodes.
Once one list is exhausted, splice the entire remainder of the other list directly.

The diagram traces merging List A (1, 4, 7) and List B (2, 3, 8): at each step the smaller current head is spliced onto the result (color-coded by source list), and once one list runs out, the rest of the other list attaches directly — node 7 from List A is simply appended whole at the end, no further comparisons needed.

6. WHEN TO USE / WHEN NOT TO USE

Use this merge technique when:

- You have two (or more, generalized) already-sorted linked lists and need a single sorted result — this is its primary, most direct use case.
- You're implementing merge sort on a linked list (Tier 1 §7) — the merge step is exactly this operation, applied recursively to halves of the original list.

Wrong choice when:

- The input lists aren't sorted — merging only works correctly because each list's current front is guaranteed to be its smallest remaining element; on unsorted input, this assumption breaks and the result won't actually be sorted.
- You're working with arrays instead of linked lists and don't need to preserve original node references — at that point, you might as well concatenate and sort directly, or use a simpler array-merge technique without the pointer-relinking overhead.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — basic merge using a dummy node (the exact trace above):

```
ListNode MergeTwoLists(ListNode a, ListNode b) {
    var dummy = new ListNode(0);
    ListNode tail = dummy;

    while (a != null && b != null) {
        if (a.Value <= b.Value) {
            tail.Next = a;
            a = a.Next;
        } else {
            tail.Next = b;
            b = b.Next;
        }
        tail = tail.Next;
    }

    tail.Next = (a != null) ? a : b; // attach whichever has leftovers
    return dummy.Next;             // skip the dummy itself
}
```

The dummy node means the very first splice (when tail is still dummy) needs no special-case logic — it's handled by the exact same `tail.Next = ...; tail = tail.Next;` pattern as every subsequent splice.

INTERMEDIATE — merging without a dummy node, requiring explicit head-tracking (showing exactly what the dummy node saves you from):

```
ListNode MergeTwoListsNoDummy(ListNode a, ListNode b) {
    if (a == null) return b;
    if (b == null) return a;

    ListNode head = (a.Value <= b.Value) ? a : b; // determine head FIRST
    ListNode tail = head;
    if (head == a) a = a.Next; else b = b.Next;

    while (a != null && b != null) {
        if (a.Value <= b.Value) { tail.Next = a; a = a.Next; }
        else { tail.Next = b; b = b.Next; }
        tail = tail.Next;
    }
    tail.Next = (a != null) ? a : b;
    return head;
}
```

Notice how much extra special-case logic is needed just to determine and set up head correctly before the main loop can even start — this is the concrete cost of not using a dummy node, making the dummy-node version (simple example) the generally preferred approach.

ADVANCED — merging k sorted lists by repeatedly applying pairwise merge (a direct preview of Tier 1 §9's heap-based optimization):

```
ListNode MergeKLists(List<ListNode> lists) {
    if (lists.Count == 0) return null;

    ListNode result = lists[0];
    for (int i = 1; i < lists.Count; i++) {
        result = MergeTwoLists(result, lists[i]); // reuse 2-list merge
    }
    return result;
}
// This works but is O(k * n) overall -- merging k lists this way means the
// growing "result" list gets re-walked and re-merged against each new list.
// Tier 1 Section 9 (Heaps) introduces a genuinely faster O(n log k) approach
// using a min-heap to always pick the smallest current head across ALL k
// lists at once, rather than merging two lists at a time, sequentially.
```

This is worth recognizing as a “correct but not optimal” solution — reusing the pairwise merge function is a perfectly valid, easy-to-explain approach, but explicitly naming that a heap-based approach (coming later in the roadmap) achieves a better time complexity shows awareness of the broader toolkit, even before you've formally learned heaps.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, the pairwise k-list merge shown above can be improved without even needing a heap yet, just by merging lists in pairs, divide-and-conquer style (merge lists 1&2, merge lists 3&4, etc., then merge those results together, repeating) — this brings the complexity down from $O(k \cdot n)$ to $O(n \log k)$ purely through better grouping of the same merge operation, foreshadowing the divide-and-conquer thinking that underlies merge sort itself (Tier 1 §7). The dummy-node technique generalizes far beyond merging too — it's a standard tool any time “handling the first element/node requires different logic than every subsequent one,” which recurs across linked list

problems generally.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is forgetting to attach the remainder of the non-exhausted list after the main comparison loop ends — without `tail.Next = (a != null) ? a : b`, you'd silently drop the rest of whichever list still had elements left, producing a shorter-than-expected, incorrect result. Another is using `<` instead of `<=` (or vice versa) when comparing equal values — this doesn't break correctness for the final sorted order, but it does affect which list's node with an equal value ends up first in the result, which matters if the problem requires stable ordering (preserving relative order of equal elements) between the two original lists. A third: assuming this merge technique works on unsorted lists “if you just compare carefully enough” — the entire algorithm's correctness depends on each list's current front being its minimum remaining element, a guarantee that simply doesn't exist for unsorted input, no amount of careful comparison logic can substitute for the lists actually being sorted beforehand.

10. SELF-CHECK

- Why does using a dummy/sentinel node eliminate the need for special-case logic when handling the very first node of the merged result?
- What specifically would go wrong with the output if you forgot the final `tail.Next = (a != null) ? a : b` line after the main comparison loop?
- Why does this merge technique fundamentally require both input lists to already be sorted — what guarantee does sortedness provide that the algorithm depends on?